

LAPORAN KEMAJUAN (MONITORING P7)

Sistem Informasi Lost & Found IPB — Progress Report Bab 1–3

Mata Kuliah : Keamanan Informasi (KOM1315)

Semester : Genap 2025/2026

Program Studi: Ilmu Komputer, IPB University

Tanggal Laporan : Mei 2026 (Pertemuan 7)

Status Laporan : Monitoring Tahap 1

Kelompok 2 :

- Syafiq Syadidul Azmi (G6401231075)
- Muhammad Faqih (G6401231081)
- Luqman Fadillah Santoso (G6401231136)

DAFTAR ISI

- [BAB 1 — PENDAHULUAN](#)
- [BAB 2 — TINJAUAN PUSTAKA](#)
- [BAB 3 — METODOLOGI & KEMAJUAN IMPLEMENTASI](#)
- [RINGKASAN PROGRESS & RENCANA SELANJUTNYA](#)

BAB 1 — PENDAHULUAN

1.1 Latar Belakang

Kampus IPB University sebagai institusi dengan puluhan ribu civitas akademika aktif setiap harinya menghadapi permasalahan umum: kehilangan barang. Berdasarkan observasi tim, penanganan barang hilang dan temuan saat ini masih dilakukan secara informal — melalui grup

media sosial, pesan WhatsApp, atau laporan lisan kepada petugas keamanan. Pendekatan ad-hoc ini menghadirkan sejumlah risiko keamanan informasi yang serius:

- **Tidak ada verifikasi identitas** bagi pelapor maupun pengklaim barang.
- **Data pribadi pengguna** (nomor telepon, foto) tersebar tanpa perlindungan kriptografis.
- **Tidak ada mekanisme audit trail** — tidak ada catatan siapa yang mengakses informasi, kapan, dan dari mana.
- **Tidak ada kendali akses** — semua pihak memiliki akses terhadap semua informasi.
- **Proses klaim tidak terverifikasi** — rentan terhadap klaim palsu.

Kondisi ini menjadikan sistem lost & found kampus sebagai *use case* yang relevan dan kaya untuk menerapkan prinsip-prinsip keamanan informasi secara komprehensif.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah yang menjadi fokus proyek ini adalah:

1. Bagaimana merancang sistem lost & found berbasis web yang menerapkan autentikasi kuat dan manajemen sesi berbasis kriptografi?
2. Bagaimana mengimplementasikan mekanisme otorisasi berbasis peran (Role-Based Access Control / RBAC) untuk membedakan hak akses antar jenis pengguna?
3. Bagaimana memastikan integritas dan kerahasiaan data pengguna melalui teknik hashing password yang aman?
4. Bagaimana membangun fondasi yang siap untuk pengembangan protokol non-repudiation dan AAA pada tahap selanjutnya?

1.3 Tujuan

Tujuan proyek ini adalah:

1. Membangun sistem web RESTful **Lost & Found IPB** yang fungsional menggunakan FastAPI dan PostgreSQL.
2. Mengimplementasikan **Password Hashing** menggunakan algoritma Argon2 dan BCrypt sebagai lapisan perlindungan kredensial pengguna.
3. Mengimplementasikan **JSON Web Token (JWT)** berbasis HMAC-SHA256 untuk autentikasi stateless dan manajemen sesi.
4. Mengimplementasikan **Role-Based Access Control (RBAC)** dengan tiga tingkat peran: UMUM, CIVITAS, dan ADMIN.

5. Menyiapkan arsitektur modular yang mendukung integrasi modul kriptografi asimetrik dan digital signature pada tahap berikutnya.

1.4 Manfaat

Mahasiswa / Civitas	Platform terpusat untuk pelaporan dan pencarian barang hilang
Administrator Kampus	Alat manajemen klaim yang terstruktur dan teraudit
Tim Pengembang	Pemahaman mendalam tentang implementasi protokol keamanan nyata
Institusi (IPB)	Peningkatan keamanan data civitas akademika

1.5 Batasan Sistem

Pada tahap monitoring P7 ini, sistem dibatasi pada:

- Implementasi backend API (belum termasuk frontend lengkap).
- Autentikasi berbasis JWT simetris (asimetrik menyusul di P9-10).
- Protokol AAA lengkap direncanakan pada P11-12.
- Sistem berjalan di lingkungan lokal menggunakan Docker Compose.

BAB 2 — TINJAUAN PUSTAKA

2.1 Keamanan Informasi dan CIA Triad

Keamanan informasi dibangun di atas tiga pilar fundamental yang dikenal sebagai **CIA Triad** :

Confidentiality (Kerahasiaan)	Informasi hanya dapat diakses oleh pihak yang berwenang	Hashing password (Argon2), JWT untuk sesi terautentikasi, RBAC
Integrity (Integritas)	Informasi tidak dapat dimodifikasi tanpa otorisasi	Validasi schema Pydantic, constraint database, HMAC pada

		JWT
Availability (Ketersediaan)	Sistem dapat diakses oleh pihak yang berwenang kapan pun dibutuhkan	Docker containerization, async FastAPI, connection pooling

2.2 Autentikasi dan Manajemen Sesi

2.2.1 JSON Web Token (JWT)

JWT (RFC 7519) adalah standar terbuka untuk mentransmisikan informasi secara aman antar pihak sebagai objek JSON yang di-sign secara digital. Token terdiri dari tiga bagian:

HEADER.PAYLOAD.SIGNATURE

Header: {"alg": "HS256", "typ": "JWT"}

Payload: {"sub": "user@ipb.ac.id", "exp": 1748700000}

Signature: HMAC-SHA256(base64url(header)+"."+base64url(payload), secret)

Keunggulan JWT untuk sistem ini:

- *Stateless* — server tidak perlu menyimpan state sesi.
- Self-contained — semua informasi yang diperlukan ada di dalam token.
- Dapat diverifikasi secara kriptografis tanpa query database pada setiap request.
- Dukungan masa berlaku (*expiry*) built-in melalui klaim `exp`.

2.2.2 OAuth 2.0 Password Flow

Sistem menggunakan OAuth2 Password Grant untuk endpoint `/token`, di mana klien mengirimkan username dan password secara langsung ke server yang tepercaya. Pola ini sesuai untuk aplikasi *first-party* di lingkungan kampus yang terkontrol.

2.3 Password Hashing

2.3.1 Argon2

Argon2 adalah algoritma hashing password pemenang *Password Hashing Competition* (PHC) 2015. Tersedia dalam tiga varian:

- **Argon2d**: Resistensi tinggi terhadap GPU/ASIC, rentan terhadap side-channel.
- **Argon2i**: Resistensi terhadap side-channel.

- **Argon2id** (*yang digunakan*): Hibrida, rekomendasi OWASP untuk general use.

Parameter yang dapat dikonfigurasi:

- **Memory Cost (m)**: Jumlah memori yang digunakan (menghambat parallelisme serangan).
- **Time Cost (t)**: Jumlah iterasi.
- **Parallelism (p)**: Jumlah thread paralel.

2.3.2 BCrypt sebagai Fallback

BCrypt dipilih sebagai algoritma fallback karena:

- Telah teruji selama lebih dari 25 tahun.
- Mendukung *work factor* yang dapat disesuaikan.
- Kompatibilitas luas dengan sistem lama.

2.4 Kontrol Akses Berbasis Peran (RBAC)

Role-Based Access Control (RBAC) adalah model kontrol akses di mana izin ditetapkan ke *peran*, dan pengguna ditetapkan ke peran tersebut. Dalam sistem ini:

```
PERAN UMUM:
- Lihat daftar dan detail barang

PERAN CIVITAS (extends UMUM):
- Melaporkan barang hilang/temuan
- Mengajukan klaim kepemilikan
- Melihat riwayat klaim sendiri

PERAN ADMIN (extends CIVITAS):
- Mereview dan memutuskan klaim
- Mengakses daftar semua pengguna
- Melihat semua klaim dari semua pengguna
```

Deteksi peran dilakukan melalui domain email pada saat registrasi:

```
if payload.email.strip().lower().endswith(("@apps.ipb.ac.id", "@ipb.ac.id")):
    resolved_role = Role.CIVITAS
```

2.5 Arsitektur Clean Architecture / Domain-Driven Design

Sistem menerapkan prinsip *Clean Architecture* (Robert C. Martin) untuk memisahkan logika bisnis dari detail implementasi:



Manfaat dari sisi keamanan:

- *Dependency inversion* memudahkan penggantian implementasi kriptografi tanpa mengubah logika bisnis.
- Pemisahan concern membatasi *blast radius* jika satu lapisan dikompromis.
- Testability lebih tinggi untuk unit test modul keamanan.

2.6 Protokol AAA (Authentication, Authorization, Accounting)

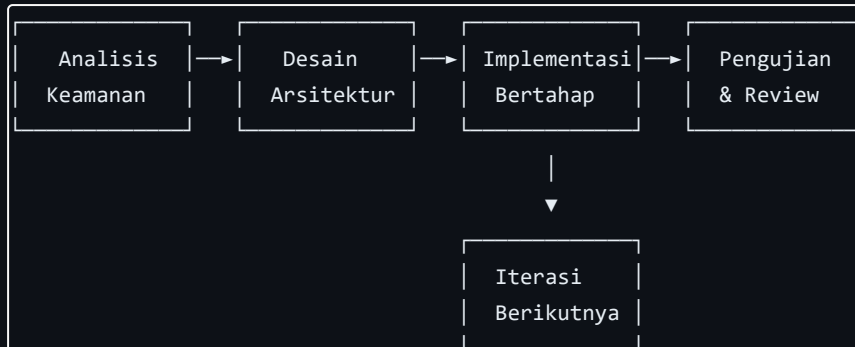
Protokol AAA adalah kerangka kerja keamanan untuk mengelola akses pengguna:

Authentication	Verifikasi identitas pengguna	✅ Diimplementasikan (JWT + Argon2)
Authorization	Menentukan apa yang boleh dilakukan	✅ Diimplementasikan (RBAC via Role enum)
Accounting	Mencatat semua aktivitas untuk audit	📅 Direncanakan (P11-12)

BAB 3 — METODOLOGI & KEMAJUAN IMPLEMENTASI

3.1 Metodologi Pengembangan

Proyek ini menggunakan pendekatan **Iterative Development** dengan prinsip *Security-by-Design*:



Prinsip yang diterapkan:

- *Least Privilege*: Setiap komponen hanya memiliki akses minimum yang diperlukan.
- *Defense in Depth*: Lapisan keamanan berlapis (middleware → auth guard → RBAC).
- *Fail Secure*: Ketika terjadi error pada validasi token, sistem menolak akses (HTTP 401).
- *Separation of Concerns*: Modul keamanan (**digital_signature**) terpisah dari logika bisnis.

3.2 Struktur Implementasi Saat Ini

Berikut adalah struktur kode yang telah diimplementasikan hingga Pertemuan 7:

```

03_Source_Code/
├── backend/
│   ├── app/
│   │   ├── api/v1/
│   │   │   ├── routers/
│   │   │   │   ├── auth_router.py    ← Redirect ke digital_signature
│   │   │   │   ├── claim_router.py   ← Endpoint klaim (protected)
│   │   │   │   ├── item_router.py    ← Endpoint barang (semi-protected)
│   │   │   │   └── user_router.py     ← Endpoint pengguna (protected)
│   │   │   └── schemas/
│   │   │       ├── auth_schema.py    ← Token response schema
│   │   │       ├── claim_schema.py   ← Claim request/response
│   │   │       ├── item_schema.py    ← Item request/response
│   │   │       └── user_schema.py    ← User request/response
│   │   ├── domains/
│   │   │   ├── claim/               ← Entity, Service, Repository
│   │   │   ├── item/               ← Entity, Service, Repository
│   │   │   └── user/               ← Entity, Service, Repository
│   │   ├── infrastructure/
│   │   │   ├── auth/               ← Auth middleware
│   │   │   ├── config/settings.py  ← Env-based configuration
│   │   │   ├── db/               ← DB configuration
│   │   │   └── repositories/      ← Concrete repositories
│   │   └── main.py               ← App entry point + middleware
  
```

```

├── seed.py                ← Database seeder
├── alembic/               ← Database migrations
├── storage/              ← File uploads (claims, items)
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
├── database/
│   ├── session.py        ← Async DB session factory
│   └── models/
│       ├── user_model.py  ← SQLAlchemy User model
│       ├── item_model.py  ← SQLAlchemy Item model
│       └── claim_model.py ← SQLAlchemy Claim model
└── digital_signature/
    ├── auth_router.py     ← Login, register, get_current_user
    ├── auth_schema.py     ← Auth schemas
    └── utils.py           ← Hashing & JWT utilities

```

3.3 Implementasi Modul Keamanan

3.3.1 Implementasi Password Hashing (`digital_signature/utils.py`)

Modul ini mengimplementasikan dua fungsi kriptografi utama untuk manajemen password:

```

from passlib.context import CryptContext

# Skema hashing berlapis: Argon2 sebagai utama, BCrypt sebagai fallback
pwd_context = CryptContext(
    schemes=["argon2", "bcrypt_sha256", "bcrypt"],
    deprecated="auto"
)

def get_password_hash(password: str) -> str:
    """Hash password plaintext menggunakan Argon2."""
    return pwd_context.hash(password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    """Verifikasi password plaintext terhadap hash yang tersimpan."""
    return pwd_context.verify(plain_password, hashed_password)

```

Keunggulan implementasi ini:

- Skema `"deprecated": "auto"` memungkinkan migrasi hash otomatis jika algoritma diperbarui.
- Password **tidak pernah** disimpan dalam bentuk plaintext.
- Hash bersifat *one-way* — tidak dapat di-reverse.

3.3.2 Implementasi JWT (digital_signature/utils.py)

```
from jose import JWTError, jwt

def create_access_token(data: Dict[str, Any], expires_delta: Optional[timedelta] = None) -> str:
    """Buat JWT access token dengan masa berlaku."""
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + (expires_delta or timedelta(minutes=60))
    to_encode.update({"exp": expire, "sub": str(data.get("sub"))})
    return jwt.encode(to_encode, settings.SECRET_KEY, algorithm=settings.ALGORITHM)

def decode_token(token: str) -> Dict[str, Any]:
    """Decode dan validasi JWT token. Raise JWTError jika tidak valid."""
    try:
        return jwt.decode(token, settings.SECRET_KEY, algorithms=[settings.ALGORITHM])
    except JWTError:
        raise
```

3.3.3 Implementasi Authentication Guard (digital_signature/auth_router.py)

```
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/token")

async def get_current_user(token: str = Depends(oauth2_scheme), db: AsyncSession =
Depends(get_db)):
    """FastAPI dependency untuk memvalidasi token dan mengambil user aktif."""
    try:
        payload = auth_utils.decode_token(token)
        username = payload.get("sub")
        if username is None:
            raise HTTPException(status_code=401, detail="Could not validate credentials")
    except Exception:
        raise HTTPException(status_code=401, detail="Could not validate credentials")

    user = await UserRepository(db).get_by_email(username)
    if not user:
        raise HTTPException(status_code=401, detail="User not found")
    return user
```

3.3.4 Implementasi RBAC (backend/app/api/v1/routers/claim_router.py)

Otorisasi berbasis peran diterapkan langsung pada endpoint yang memerlukan hak akses khusus:

```
@router.patch("/claims/{claim_id}/review")
async def review_claim(
    claim_id: int,
    payload: ReviewClaimRequest,
    db: AsyncSession = Depends(get_db),
    current_user: UserEntity = Depends(get_current_user) # ← Auth guard
):
    # ← Authorization check: hanya ADMIN yang boleh review klaim
    if current_user.role != Role.ADMIN:
        raise HTTPException(
            status_code=403,
            detail="Hanya admin yang dapat mereview claim"
        )
    # ... logika review
```

Untuk klaim, pengguna non-admin hanya bisa melihat klaim miliknya sendiri:

```
@router.get("/claims/")
async def list_claims(..., current_user: UserEntity = Depends(get_current_user)):
    # Pembatasan data: non-admin hanya melihat klaim sendiri
    if current_user.role != Role.ADMIN:
        claimer_id = current_user.id # ← Override parameter dengan ID sendiri
    # ...
```

3.3.5 Implementasi Registrasi dengan Deteksi Peran Otomatis

```
@router.post("/users/")
async def create_user(payload: CreateUserRequest, ...):
    # Auto-detect role berdasarkan domain email IPB
    resolved_role = Role.UMUM
    if payload.email.strip().lower().endswith(("@apps.ipb.ac.id", "@ipb.ac.id")):
        resolved_role = Role.CIVITAS

    user_data = UserEntity(
        email=payload.email,
        fullname=payload.fullname,
        password_hashed=auth_utils.get_password_hash(payload.password), # ← Hash saat
registrasi
        role=resolved_role,
        is_active=True,
    )
```

3.4 Skema Database

Skema database yang telah diimplementasikan dan dimigrasi menggunakan Alembic:

Tabel `users`

<code>id</code>	INTEGER	PRIMARY KEY, INDEX	Auto-increment ID
<code>email</code>	VARCHAR	UNIQUE, NOT NULL	Digunakan sebagai username
<code>full_name</code>	VARCHAR	NOT NULL	Nama lengkap
<code>is_active</code>	BOOLEAN	DEFAULT TRUE	Status akun
<code>password_hash</code>	VARCHAR	NOT NULL	Hash Argon2/BCrypt — tidak pernah plaintext
<code>role</code>	ENUM	DEFAULT 'UMUM'	UMUM CIVITAS ADMIN

Tabel `items`

<code>id</code>	INTEGER	PRIMARY KEY	Auto-increment ID
<code>title</code>	VARCHAR(100)	NOT NULL	Judul barang
<code>description</code>	TEXT	NOT NULL	Deskripsi detail
<code>location</code>	VARCHAR(255)	NOT NULL	Lokasi kejadian
<code>image</code>	VARCHAR(255)	NULLABLE	Path file foto
<code>latitude</code>	FLOAT	NULLABLE	Koordinat GPS
<code>longitude</code>	FLOAT	NULLABLE	Koordinat GPS
<code>category</code>	VARCHAR(64)	NULLABLE	Kategori barang
<code>report_type</code>	ENUM	NOT NULL	LOST FOUND
<code>status</code>	ENUM	DEFAULT 'LOST'	LOST FOUND CLAIMED
<code>created_at</code>	DATETIME	NOT NULL	Timestamp pelaporan
<code>reporter_id</code>	INTEGER	FK(users.id)	Relasi ke pelapor

Tabel **claim**

id	INTEGER	PRIMARY KEY	Auto-increment ID
proof_text	TEXT	NOT NULL	Bukti teks klaim
proof_image	VARCHAR(255)	NOT NULL	Path foto bukti
status	ENUM	DEFAULT 'PENDING'	PENDING APPROVED REJECTED
created_at	DATETIME	NOT NULL	Timestamp klaim
updated_at	DATETIME	NULLABLE	Timestamp update
reviewed_at	DATETIME	NULLABLE	Timestamp review
claimer_id	INTEGER	FK(users.id)	Relasi ke pengklaim
item_id	INTEGER	FK(items.id)	Relasi ke barang
reviewer_id	INTEGER	FK(users.id), NULLABLE	Relasi ke reviewer (admin)

3.5 Endpoint API yang Telah Diimplementasikan

POST	/users/	✗	-	Registrasi pengguna baru
POST	/token	✗	-	Login & dapatkan JWT token
GET	/users/me	✓ JWT	Any	Profil pengguna yang login
GET	/users/	✓ JWT	Any	Daftar semua pengguna
GET	/users/{id}	✓ JWT	Any	Detail pengguna
GET	/items/	✗	-	Daftar semua barang
GET	/items/{id}	✗	-	Detail barang

POST	/items/report-lost	✓ JWT	CIVITAS+	Laporkan barang hilang
POST	/items/report-found	✓ JWT	CIVITAS+	Laporkan barang temuan
POST	/claims/	✓ JWT	CIVITAS+	Ajukan klaim kepemilikan
GET	/claims/	✓ JWT	ADMIN (all) / User (own)	Daftar klaim
GET	/claims/{id}	✓ JWT	Any	Detail klaim
PATCH	/claims/{id}/review	✓ JWT	ADMIN only	Review dan putuskan klaim

3.6 Konfigurasi Keamanan

Seluruh konfigurasi sensitif dikelola melalui environment variable dan tidak pernah di-hardcode atau di-commit ke repositori:

```
# backend/.env (terdaftar di .gitignore)
DATABASE_URL=postgresql+asyncpg://user:password@localhost:5432/lostnfound
SECRET_KEY=<minimum-256-bit-random-secret>
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
CLAIM_UPLOAD_DIR=backend/storage/claims
```

```
# backend/.env.example (safe to commit – tanpa nilai sensitif)
DATABASE_URL=postgresql+asyncpg://user:password@localhost:5432/dbname
SECRET_KEY=your-secret-key-here
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
CLAIM_UPLOAD_DIR=backend/storage/claims
```

File `.gitignore` dikonfigurasi untuk mencegah commit kunci kriptografi:

```
.env
*.env
*.key
*.pem
```

3.7 Deployment dengan Docker Compose

Sistem dapat dijalankan secara konsisten di berbagai lingkungan menggunakan Docker:

```
# docker-compose.yml (ringkasan)
services:
  backend:
    build: .
    command: uvicorn backend.app.main:app --host 0.0.0.0 --port 8000
    environment:
      - DATABASE_URL=...
      - SECRET_KEY=...
    depends_on:
      - db

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: lostnfound
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
```

3.8 Progress Checklist Hingga P7

Inisiasi repositori Git	✓ Selesai	Struktur folder sesuai ketentuan
Proposal Teknis	✓ Selesai	Lihat <code>01_Proposal_&_Analisis/</code>
Identifikasi Aset & Threat Model	✓ Selesai	Terdokumentasi dalam proposal
Desain Arsitektur Sistem	✓ Selesai	Clean Architecture + security layers
Skema Database (ERD)	✓ Selesai	3 tabel utama: users, items, claim
Migrasi Database (Alembic)	✓ Selesai	Versi migrasi terkontrol
CRUD Barang (Items)	✓ Selesai	Report lost, found, list, detail
CRUD Klaim (Claims)	✓ Selesai	Submit, list, detail, review
Manajemen Pengguna	✓ Selesai	Register, list, detail
Password Hashing (Argon2)	✓ Selesai	Modul <code>digital_signature/utils.py</code>
JWT Authentication	✓ Selesai	HMAC-SHA256, 60 menit expiry
RBAC (Role-Based Access)	✓ Selesai	3 peran: UMUM, CIVITAS, ADMIN
Proteksi Secret Key (<code>.gitignore</code>)	✓ Selesai	<code>.env</code> tidak pernah di-commit

Containerization (Docker)	✔ Selesai	<code>docker-compose.yml</code> tersedia
Tanda Tangan Digital	📅 Planned	Target: P9-10
AAA Accounting (Logging)	📅 Planned	Target: P11-12
Unit Testing Modul Keamanan	📅 Planned	Target: P13

RINGKASAN PROGRESS & RENCANA SELANJUTNYA

Ringkasan Pencapaian (Minggu 1–7)

Hingga Pertemuan 7, tim telah berhasil membangun **fondasi sistem yang solid** dengan seluruh fungsionalitas inti dan lapisan keamanan dasar telah berjalan:

- Infrastruktur** — Sistem berjalan penuh menggunakan Docker Compose dengan backend FastAPI dan database PostgreSQL.
- Authentication** — JWT berbasis HMAC-SHA256 dengan masa berlaku 60 menit, diintegrasikan menggunakan `OAuth2PasswordBearer`.
- Password Security** — Hashing Argon2id berlapis dengan BCrypt sebagai fallback melalui `passlib.CryptContext`.
- Authorization (RBAC)** — Tiga tingkat peran dengan penegakan otorisasi pada setiap endpoint yang sensitif.
- Secret Management** — Seluruh konfigurasi sensitif dikelola melalui environment variable, tidak ada hardcoded key di repositori.
- Domain Architecture** — Pemisahan bersih antara router, service, repository, dan model mengikuti prinsip *Clean Architecture*.

Kendala yang Dihadapi

Kompleksitas async SQLAlchemy untuk relasi antar tabel	Menggunakan <code>selectinload</code> untuk eager loading yang aman
Sinkronisasi antara modul <code>digital_signature</code> dan <code>backend</code>	Menjalankan dari root <code>03_Source_Code</code> agar semua paket terdeteksi

Handling upload file bersamaan dengan form data di FastAPI	Menggunakan <code>Form(...)</code> dan <code>File(...)</code> secara eksplisit
--	--

Rencana Tahap Selanjutnya (P8–P12)

P8	Refinement	Review kode, penambahan validasi edge case
P9	Asymmetric Crypto	Implementasi RSA/ECDSA untuk tanda tangan digital
P10	Digital Signature	Non-repudiation pada transaksi klaim
P11	AAA Accounting	Logging aktivitas user ke database (audit trail)
P12	Integration Testing	End-to-end test, uji penetrasi sederhana
P13	Unit Testing	Testing modul keamanan, dokumentasi hasil
P14-15	Final Release	Paper ilmiah, user manual, demo video

Kontribusi Tim

(Nama Anggota 1)	Arsitektur sistem, <code>main.py</code> , Docker setup	-
(Nama Anggota 2)	Modul <code>digital_signature</code> , JWT, Hashing	-
(Nama Anggota 3)	Database models, Alembic migrations	-
(Nama Anggota 4)	Domain services, claim & item logic	-
(Nama Anggota 5)	Dokumentasi, seed data, testing awal	-

Catatan : Isi nama anggota, NIM, dan jumlah commit aktual sesuai dengan data dari repositori GitHub kelompok.

Laporan ini merupakan luaran Monitoring Tahap 1 (Pertemuan 7) dari mata kuliah Keamanan Informasi (KOM1315), Program Studi Ilmu Komputer, IPB University, Semester Genap 2025/2026.

Dokumen dibuat berdasarkan implementasi aktual pada repositori:
KOM1315_SmtGenap26_KelompokXX_LostFoundIPB